ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ



БИЕ ДААЛТ 1

ГАРЫН АВЛАГА

F.CSM302 Үйлдлийн систем
2026–2027 оны хичээлийн жилийн намар

Сэдэв:	The Producer–Consumer Problem. Programming Project 4 (7-р бүлэг), Buffer animation
Хуваарилалтын код:	V-1-12
Хичээл заасан багш:	Г. Ганбат /F.CS10/
Гүйцэтгэсэн баг:	Н. Баасандорж /B241910012/ ..... /...../ ..... /...../ ..... /...../ ..... /...../
Repository:	`github.com/anunbayar1/CSM302-2627-BD1`
Огноо:	2026 оны 10 дугаар сарын 27

Улаанбаатар хот, 2026

ГАРЧИГ

НЭГ. ХЭРЭГЛЭГЧИЙН ЗААВАР **2**

1.1 Программын зорилго 2

1.2 Дэмжих орчин 3

1.3 Суулгах, нээх 3

1.4 Windows дээр 3

1.5 Нүүр хуудас 4

1.6 Шуудангийн хайрцаг (Python · жишээтэй) 4

1.7 C · Pthreads хуудас 5

1.8 Ороолт ба тохиргоо 6

1.9 Ажиллуулах, зогсоох, reset хийх 7

1.10 Дүрслэлийн тэмдэг, өнгө, төлөв 8

1.11 Алдааны туршилт 8

1.12 Жишээ 10

1.13 Лог татах 12

1.14 Түгээмэл алдаа ба шийдэл 13

ХОЁР. OPERATIONAL ЗААВАР **14**

2.1 Repository clone 14

2.2 OS, compiler/runtime, version 14

2.3 Dependency суулгах 15

2.4 Build ба Run 15

2.5 Үндсэн тестийн өгөгдөл 15

2.6 Нэмэлт boundary ба error тест 16

2.7 Хүлээгдсэн үр дүн 17

2.8 Known issues 17

2.9 Нөөц ба эх сурвалж 18

НЭГ. ХЭРЭГЛЭГЧИЙН ЗААВАР

1.1 Программын зорилго

Сурах бичгийн (Silberschatz, Galvin, Gagne, *Operating System Concepts*, 10-р хэвлэл) 7-р бүлгийн Programming Project 4 буюу Producer–Consumer (bounded buffer) бодлогыг C хэл дээр POSIX Pthreads ашиглан хэрэгжүүлсэн. Producer thread-үүд санамсаргүй тоо (item) үүсгэж хязгаартай тойрог буферт тавьдаг. Consumer thread-үүд буферээс авдаг. Буферийг гурван синхрончлолын хэрэгслээр хамгаална:

- `empty` : хоосон нүдний тоог тоолдог семафор (эхлэх утга = BUFFER_SIZE). Буфер дүүрэн үед producer энэ дээр хүлээнэ.
- `full` : дүүрэн нүдний тоог тоолдог семафор (эхлэх утга = 0). Буфер хоосон үед consumer энэ дээр хүлээнэ.
- `mutex` : `pthread_mutex_t` . Нэг удаад ганцхан thread буфер ба `in` / `out` индексийг өөрчилнө.

Браузер дээрх дэлгэц нь бодит C программын thread бүрийн үйлдлийг (унтах, семафор хүлээх, mutex хүлээх, бичих, унших) буферийн animation-оор алхам алхмаар харуулна. Бүх синхрончлол C программ дотор явагддаг. Дэлгэц зөвхөн харуулж, удирдах үүрэгтэй. Нүүр хуудаснаас хоёр горимын аль нэгийг сонгоно:

- **Шуудангийн хайрцаг (Python · жишээтэй):** шуудан зөөгч (producer), хэрэглэгч (consumer), шуудангийн хайрцаг (buffer)-ийн жишээгээр, монгол дуут тайлбартайгаар ойлгомжтой тайлбарласан горим. Хөдөлгүүр нь ижил алгоритмыг Python `threading` -ээр хэрэгжүүлсэн `bounded_buffer.py` .
- **Шууд ажиллагаа (C · Pthreads):** номын C программын бодит ажиллагаа: producers → buffer → consumers. Семафор, mutex-ийн бодит утга, thread бүрийн төлөв, лог болон алдааны туршилтыг харуулна.

1.2 Дэмжих орчин

Хүснэгт 1. Дэмжих орчин

Хэсэг	Шаардлага
Үйлдлийн систем	Linux (Debian, Ubuntu 22.04+). Windows дээр WSL2 доторх Ubuntu. Windows дээр шууд ажиллуулбал зөвхөн Python горим ажиллана. macOS дээр туршаагүй.
C хэсэг	`gcc` (C11), GNU `make`, POSIX Pthreads ба семафор (`sem_timedwait`, `pthread_mutex_timedlock`)
Веб сервер	Python 3 (зөвхөн стандарт сан, pip сан шаардлагагүй)
Браузер	Chrome, Edge, Firefox-ийн сүүлийн хувилбар. Дэлгэц 1280×720 ба түүнээс том үед нэг дэлгэцэд багтана.
Дуут тайлбар (сонголттой)	Аудио файл үүсгэхэд `edge-tts` ба интернет хэрэгтэй. Аудио байхгүй үед хөтчийн монгол хоолой эсвэл зөвхөн хадмал ашиглана.

1.3 Суулгах, нээх

Debian / Ubuntu дээр:

```
sudo apt update
sudo apt install git build-essential python3
git clone git@github.com:anunbayar1/CSM302-2627-BD1.git
cd CSM302-2627-BD1
python3 src/gui.py
```

`gui.py` асахдаа C программыг (`src/c/producer_consumer`) автоматаар дахин хөрвүүлнэ, дараа нь хөтчөөр `http://127.0.0.1:8765/` хаягийг нээнэ. Хөтөч өөрөө нээгдэхгүй бол терминал дээр хэвлэгдсэн хаягийг гараар нээнэ. Порт эзэлсэн байвал дараагийн чөлөөт портыг (8766, 8767…) автоматаар сонгоно. Серверийг терминал дээр `Ctrl+C` дарж зогсооно.

```
$ python3 src/gui.py
Нүүр хуудас:            http://127.0.0.1:8765/
  Python · шуудан:      http://127.0.0.1:8765/mail
  C · Pthreads · шууд:  http://127.0.0.1:8765/raw
Браузер нээгдэхгүй бол дээрх хаягийг өөрөө нээнэ үү. Зогсоох: Ctrl+C
```

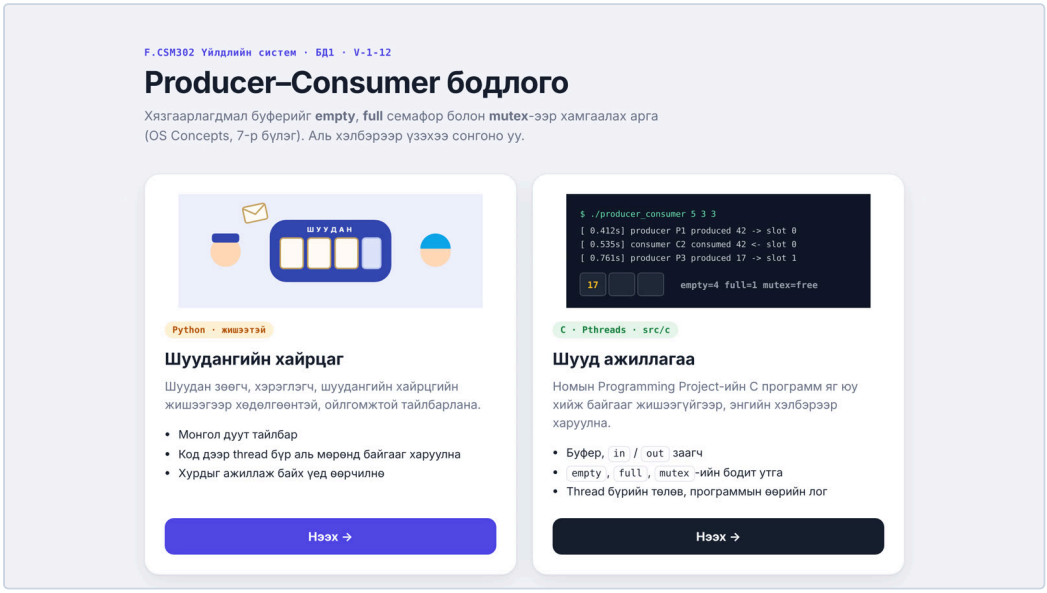
1.4 Windows дээр

A. WSL2 (бүрэн ажиллагаа). PowerShell-ийг администратораар нээж `wsl --install -d Ubuntu` гэж ажиллуулаад компьютероо дахин асаана. Ubuntu терминалыг нээж, 1.3-ын командуудыг яг адилхан ажиллуулна. `http://127.0.0.1:8765/` хаягийг Windows-ийн хөтчөөр нээнэ.

Б. Шууд Windows дээр (зөвхөн Python горим). python.org-оос Python 3 суулгана (*Add python.exe to PATH*-ийг чагтална). Repository-г татаад PowerShell дээр `py src\gui.py` гэж ажиллуулна. C программ POSIX семафор ашигладаг тул энэ үед C · Pthreads хуудас ажиллахгүй, нүүр хуудсанд шар анхааруулга гарна.

1.5 Нүүр хуудас

Нүүр хуудсанд хоёр карт байна (Зураг 1). **Шуудангийн хайрцаг** карт `/mail` хуудас руу, **Шууд ажиллагаа** карт `/raw` хуудас руу оруулна. Хуудас бүрийн зүүн дээд буланд байрлах ← товчоор нүүр хуудас руу буцна. Хоёр хуудсыг хоёр цонхонд зэрэг нээж харьцуулж болно.

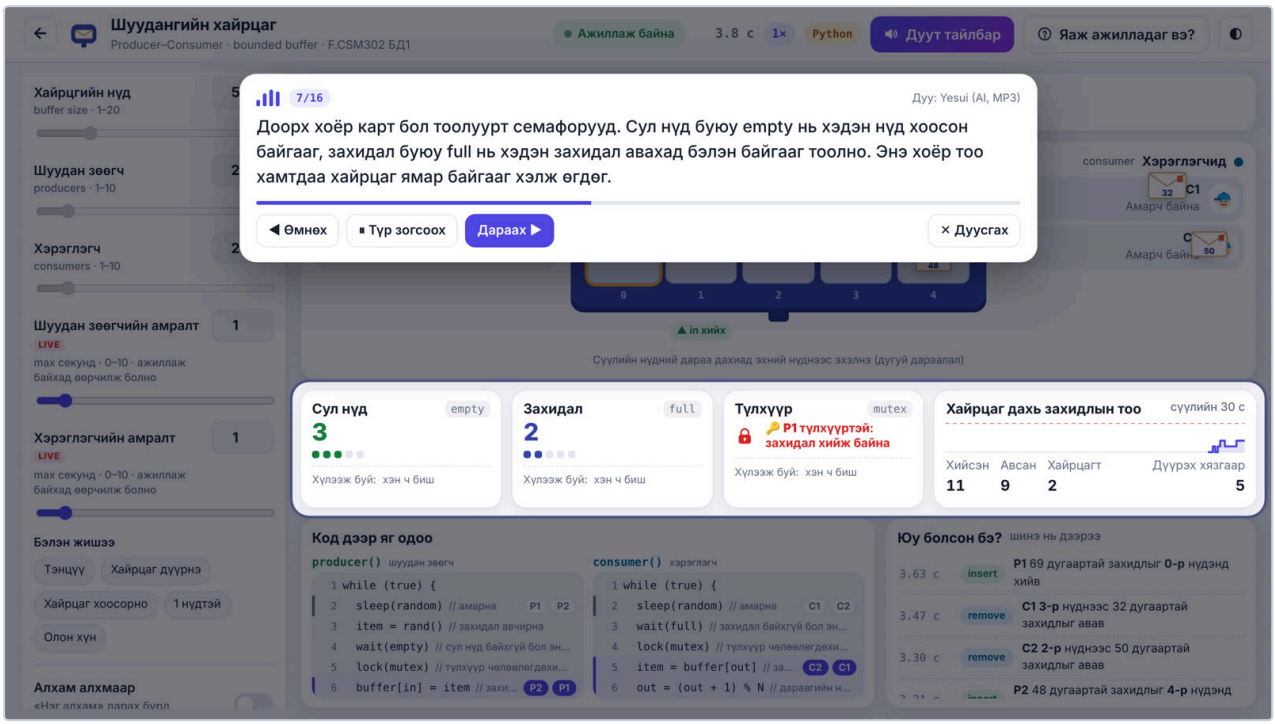


Зураг 1. Нүүр хуудас: горим сонгох

1.6 Шуудангийн хайрцаг (Python · жишээтэй)

Зүүн баганад **Тохиргоо** ба **Удирдлага** байна. Голд шуудан зөөгчид (producer), шуудангийн хайрцаг (buffer), хэрэглэгчид (consumer) байрлана. Доор нь **Сул нүд** (`empty`), **Захидал** (`full`), **Түлхүүр** (`mutex`) картууд болон хайрцаг дахь захидлын тооны график, **Код дээр яг одоо** (producer/consumer кодын аль мөрөнд thread бүр байгаа), **Юу болсон бэ?** (үйл явдлын жагсаалт) хэсэг байна.

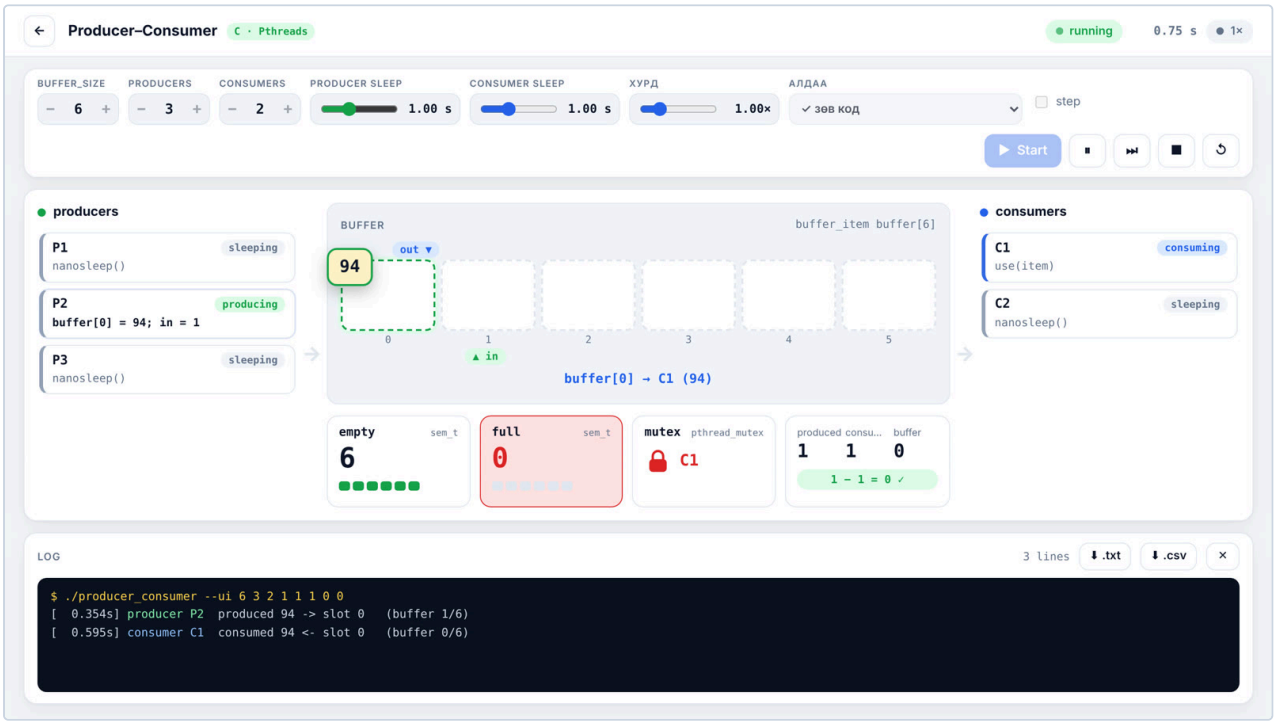
- **Дуут тайлбар:** баруун дээд буланд байрлах товч. 16 алхамтай тайлбар эхэлнэ. Алхам бүрт тайлбарлаж буй хэсэг тодорч, бусад хэсэг бүдгэрнэ. Доор нь хадмал гарч, монгол хоолойгоор уншина (Зураг 2). **◀ Өмнөх**, **■ Түр зогсоох**, **Дараах ▶**, **✕ Дуусгах** товчтой. Тайлбар явцдаа симуляцыг өөрөө эхлүүлж, буфер дүүрэх, хоосрох, хурд өөрчлөхийг харуулна.
- **Яаж ажилладаг вэ?**: товч тайлбар бүхий цонх. Хажууд нь цайвар/бараан өнгө солих товч байна.
- **Бэлэн жишээ:** Тэнцүү, Хайрцаг дүүрнэ, Хайрцаг хоосорно, 1 нүдтэй, Олон хүн. Дарахад тохиргоог бөглөнө.



Зураг 2. Дуут тайлбарын 7-р алхам: семафорын картууд тодорсон байдал

1.7 C · Pthreads хуудас

Дээд мөрөнд тохиргоо ба удирдлагын товч байна. Голд нь зүүнээс баруун тийш **producers** → **buffer** → **consumers** гэж урсгалын дагуу байрлана (Зураг 3). Буферийн доор `empty`, `full` семафор, `mutex` болон produced / consumed / buffer тоолуур, хамгийн доор C программын терминалын гаралттай ижил форматтай лог байна.



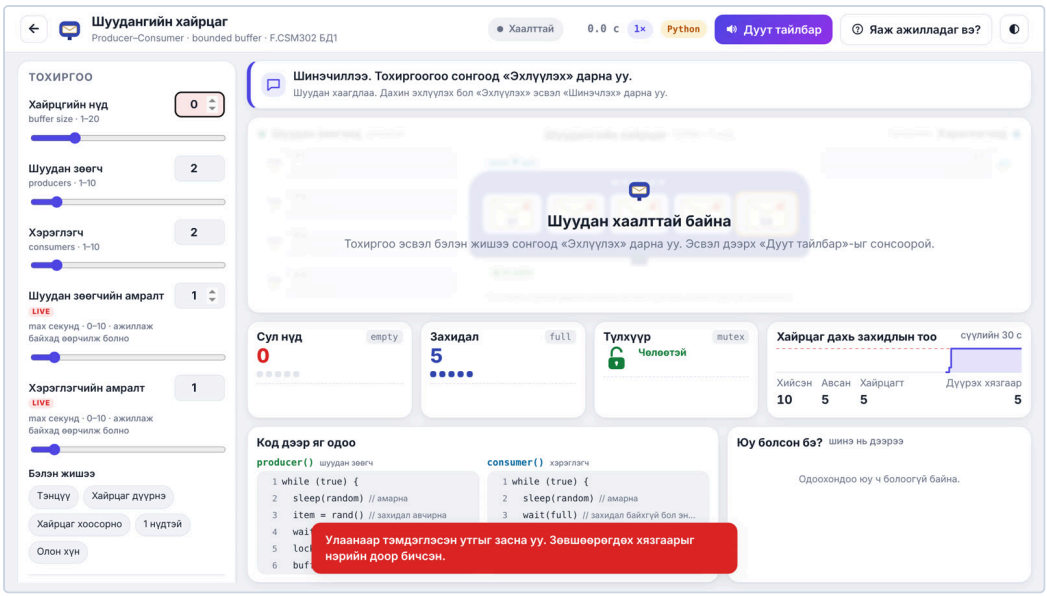
Зураг 3. C · Pthreads хуудас: P2-ийн бичсэн item 94 буферээс C1 үүг нисэж байна

1.8 Оролт ба тохиргоо

Хүснэгт 2. Оролт ба тохиргоо

Талбар	Утга	Хүрээ	Анхдагч	Ажиллаж байх үед
BUFFER_SIZE / Хайрцгийн нүд	тойрог буферийн нүдний тоо	1–20	5	өөрчлөхгүй
producers / Шуудан зөөгч	producer thread-ийн тоо	1–10	2	өөрчлөхгүй
consumers / Хэрэглэгч	consumer thread-ийн тоо	1–10	2	өөрчлөхгүй
producer sleep / амралт	producer үйлдэл хооронд 0-ээс энэ хүртэл санамсаргүй хугацаа унтана (сек)	C: 0–3 (алхам 0.05) Python: 0–10 (алхам 0.1)	1.0	шууд өөрчлөгдөнө
consumer sleep / амралт	consumer-ийн мөн адил унтах хугацаа (сек)	адил	1.0	шууд өөрчлөгдөнө
хурд	бүх хугацааг үржүүлэх коэффициент	C: 0.25–4× (алхам 0.05) Python: 0.25, 0.5, 1, 2, 4×	1×	шууд өөрчлөгдөнө
step / Алхам алхмаар	чагталбал Pause төлөвт эхэлж, Step бүрт нэг үйлдлээр урагшилна	тийм / үгүй	үгүй	—
алдаа (зөвхөн C)	зөв кодоос нэг алдаа гаргаж туршина (1.11-ийг үзнэ үү)	5 сонголт	✓ зөв код	өөрчлөхгүй

Python хуудсанд хүрээнээс гадуур утга оруулбал талбар улаан болж, «Эхлүүлэх» дарахад алдааны мэдээлэл гарна (Зураг 4). Сервер ч мөн адил шалгаж, монгол хэлээр алдааны мэдээлэл буцаана. C хуудасны − / + товч утгыг хүрээнээс гаргахгүй. Гараар буруу утга бичвэл сервер мөн алдааны мессеж буцаана.



Зураг 4. Буруу оролт: хайрцгийн нүд = 0 үед

1.9 Ажиллуулах, зогсоох, reset хийх

Хүснэгт 3. Удирдлагын товч

Товч (C / Python)	Үүрэг
▶ Start / Эхлүүлэх	Тохиргоогоор шинэ run эхлүүлнэ: C программыг `--ui` горимоор (эсвэл Python хөдөлгүүрийг) ажиллуулж thread-үүдийг үүсгэнэ.
⏸ Pause / Түр зогсоох	Thread-үүд дараагийн шалгах цэг дээр зогсоно. Дахин дарахад (▶ Resume) үргэлжилнэ.
⏭ Step / Нэг алхам	Түр зогссон үед нэг thread-ийг нэг алхмаар урагшлуулна.
■ Stop / Зогсоох	Бүх thread-ийг зогсоож join хийнэ. Эцсийн дүнг (`CONSISTENT` / `INCONSISTENT` / `DEADLOCK`) лог дээр хэвлэнэ. Лог хадгалагдсан хэвээр үлдэнэ.
↺ Reset / Шинэчлэх	Run-ийг зогсоож, дэлгэц болон логийг цэвэрлэнэ.
⬇ .txt, ⬇ .csv, ✕	Логийг файлаар татах, цэвэрлэх (1.13).

1.10 Дүрслэлийн тэмдэг, өнгө, төлөв

Хүснэгт 4. C хуудасны тэмдэглэгээ

Тэмдэг	Утга
Тасархай хүрээтэй нүд	хоосон нүд
Шар нүд, дотор нь тоо	item-тэй нүд
Ногоон тасархай, анивчсан нүд	item нисэж орох гэж буй нүд
Нисэж буй тоо	бичих: producer → `buffer[in]` ; унших: `buffer[out]` → consumer
`out ▼`, `▲ in`	consumer-ийн унших, producer-ийн бичих дараагийн нүд. `(i + 1) % N` -ээр гулсаж шилжинэ.
Thread карт: sleeping / producing / consuming	унтаж байна (`nanosleep()`) / item үүсгэж байна / item ашиглаж байна
Шар карт: waiting	хүлээж байна. Код мөрөнд `sem_wait(&empty)` , `sem_wait(&full)` эсвэл `pthread_mutex_lock(&mutex)` гэж аль дээр хүлээж байгааг харуулна.
Карт дээрх тод мөр	яг гүйцэтгэж буй мөр, жишээ нь `buffer[2] = 57; in = 3`
Улаан тэгтэй семафорын карт	утга 0: тэр семафор дээр thread блоклогдоно
mutex карт: 🔒 P1 / free / ⏳ P2 C1	mutex-ийг хэн барьж байна / чөлөөтэй / mutex хүлээж буй thread-үүд
`13 − 9 = 4 ✓`	produced − consumed = буфер дахь item: өгөгдөл алдагдаагүй
Улаан нүд, улаан карт, ⚠ N, × lost / garbage	алдааны туршилтад илэрсэн алдаа (1.11)
⛔ DEADLOCK	бүх thread бие биеэ хүлээж түгжирсэн

Шуудангийн хуудсанд мөн адил: шар карт = хүлээж байна, 🔑 = хайрцгийн түлхүүр (mutex) эзэмшиж байна, захидал нисэж хайрцагт орох/гарах, «Код дээр яг одоо» хэсэгт thread бүрийн нэр тухайн мөрөн дээр харагдана.

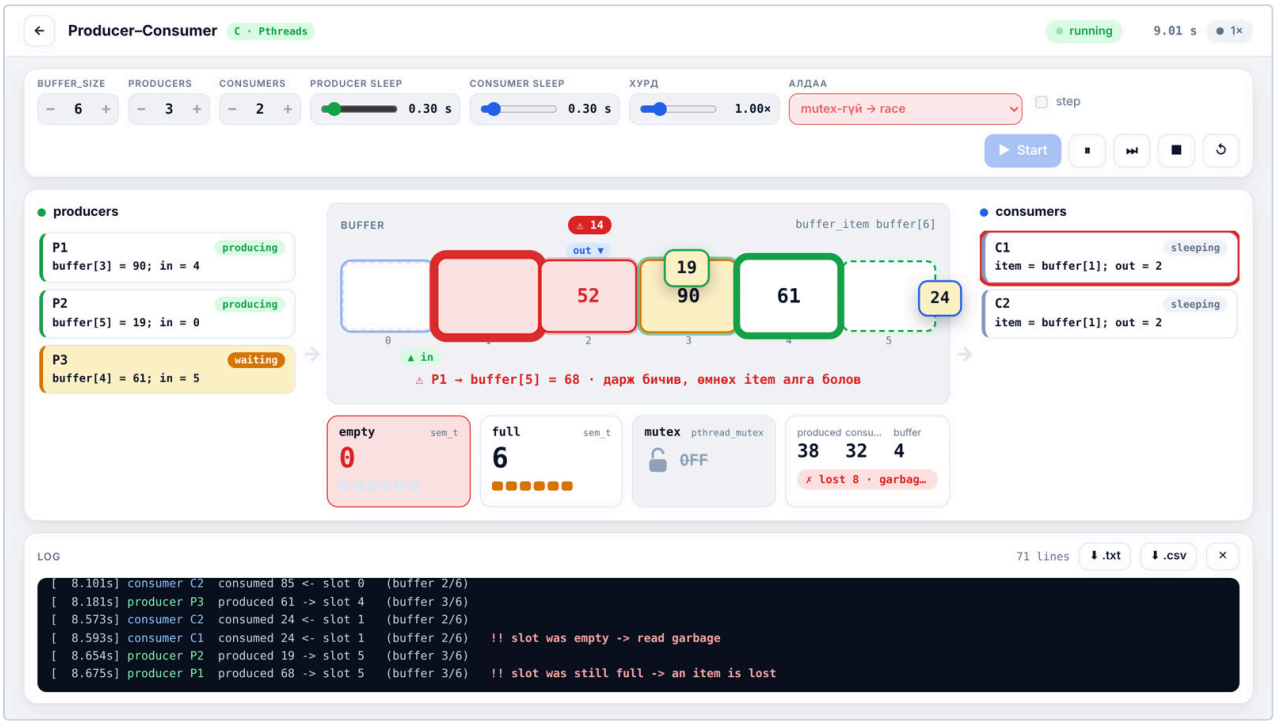
1.11 Алдааны туршилт

C хуудасны **алдаа** сонголт (терминал дээр `--bug`) нь зөв кодоос нэг сонгодог алдааг гаргаж, юу болохыг харуулна. Нүд бүрт item байгаа эсэхийг атомар шалгагчаар хянаж, алдаа бүрийг илрүүлнэ. Энэ шалгагч алгоритмын хэсэг биш, зөвхөн хэмжилт.

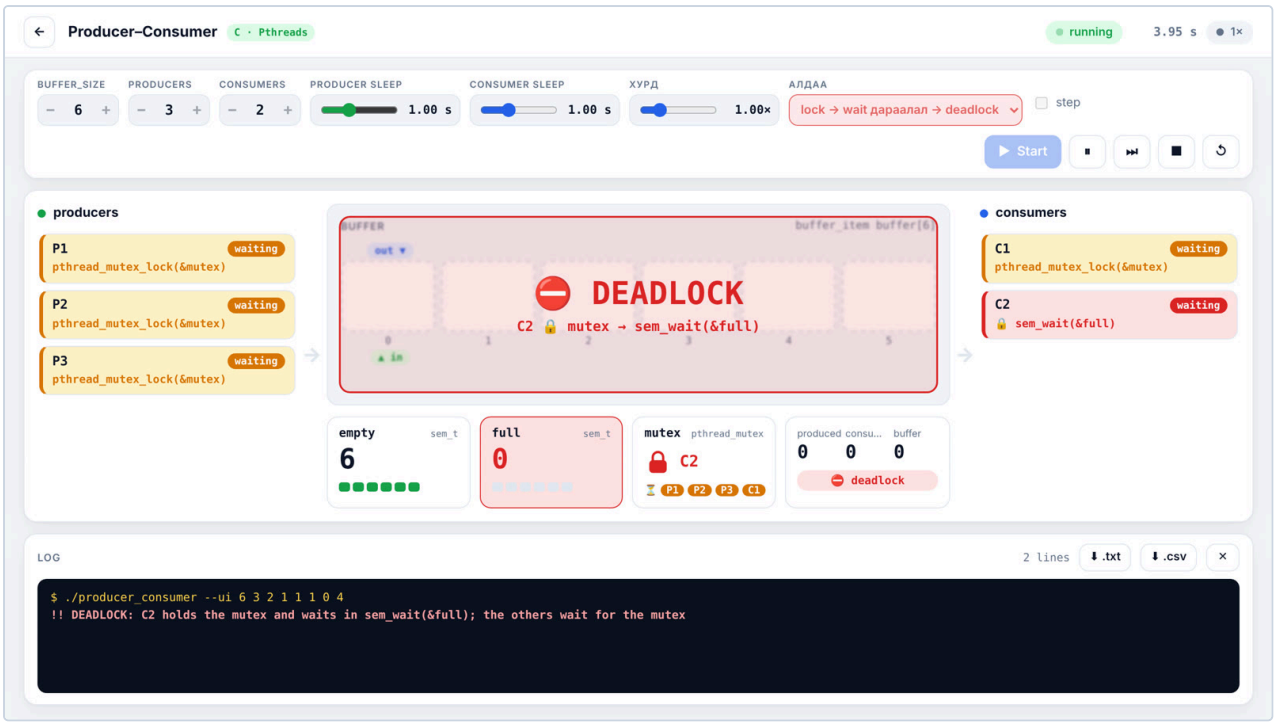
Хүснэгт 5. Алдааны горимууд

Сонголт	Кодын алдаа	Үр дагавар ба дэлгэц
✓ зөв код	—	Алдаа гарахгүй, `✓`
mutex-гүй → race	`pthread_mutex_lock/unlock` байхгүй	Race condition: хоёр thread нэг `in` утгыг уншиж нэг нүд рүү бичнэ. Item алга болох (*lost*), хоосон нүднээс унших (*garbage*) явдал гарна.
wait(empty)-гүй → overflow	producer `sem_wait(&empty)` хийхгүй	Буфер дүүрсэн ч producer бичсээр, уншаагүй item дээр дарж бичнэ.
wait(full)-гүй → underflow	consumer `sem_wait(&full)` хийхгүй	Буфер хоосон байхад consumer уншина. Буферийн тоолуур сөрөг болно.
lock → wait дараалал → deadlock	`mutex_lock`-ийг `sem_wait`-оос өмнө хийнэ	Mutex барьсан thread семафор хүлээж, бусад нь mutex хүлээнэ. Хэн ч урагшлахгүй, ⛔ DEADLOCK.

Mutex-гүй горимд race condition-ийг харагдахуйц болгохын тулд critical section дотор 40 мс-ийн (терминалд 0.1 мс) завсар нэмсэн. Энэ нь тэр мөчид өөр thread тасалдуулсныг (preemption) дуурайлгана.



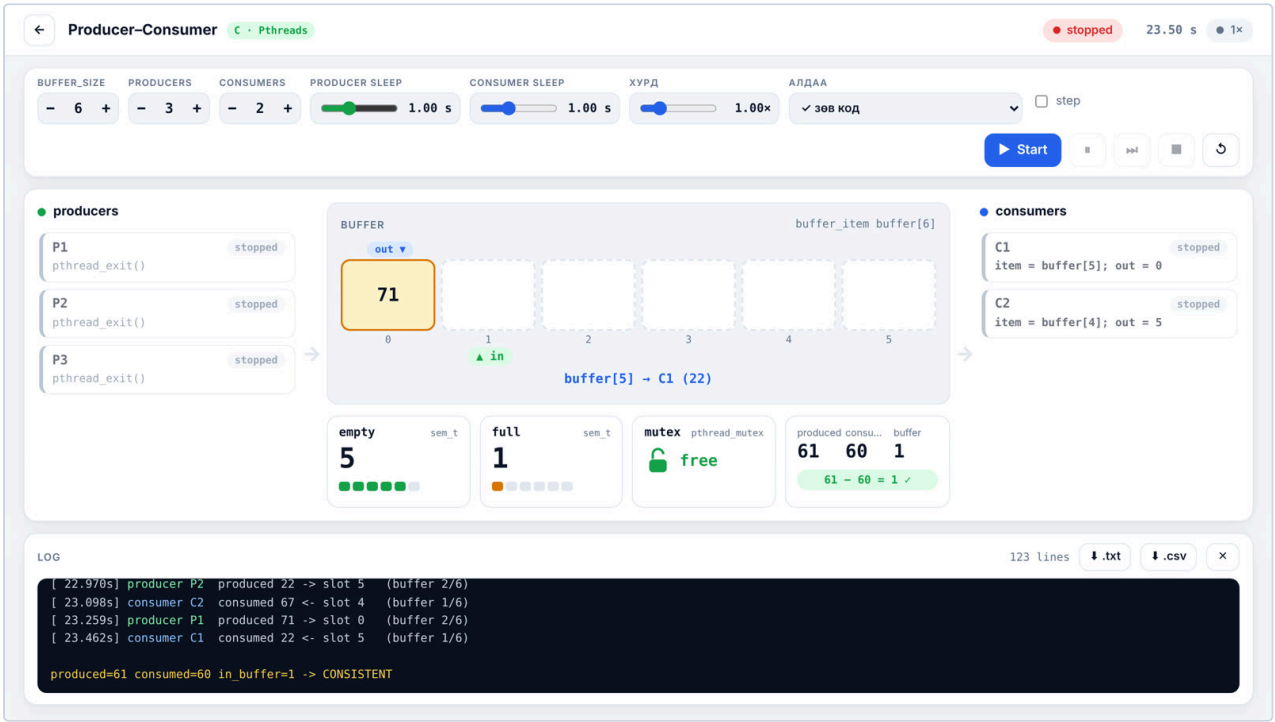
Зураг 5. Mutex-гүй үед: P1 дүүрэн нүд рүү дарж бичиж, өмнөх item алга болсон (улаан нүд)



Зураг 6. Deadlock: C1 mutex-ийг барьсаар `sem_wait(&full)` дээр хүлээж, бусад нь mutex хүлээж байна

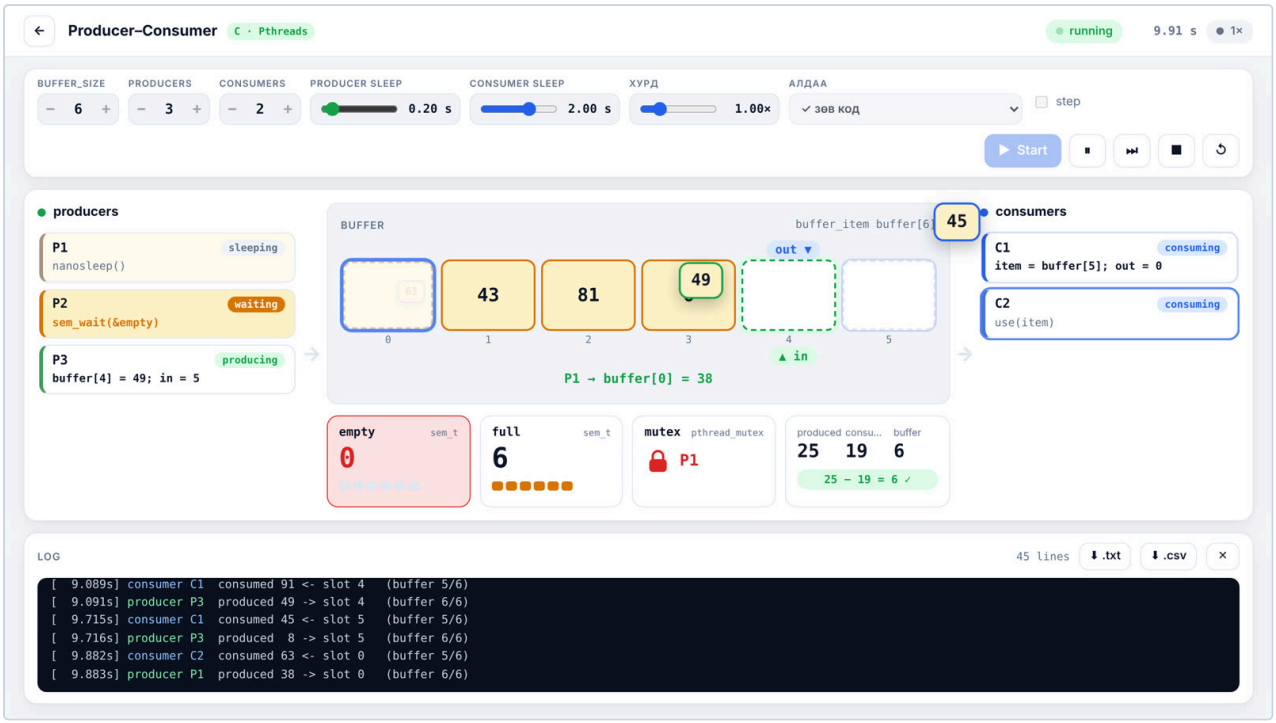
1.12 Жишээ

Жишээ 1. Хэвийн ажиллагаа (C, BUFFER_SIZE = 6, producer 3, consumer 2, sleep 1 с). Start дарахад item producer-оос буфер рүү, буферээс consumer руу нисэж, `in` / `out` заагч гулсаж шилжинэ. Stop дарахад шалгалт `61 − 60 = 1 ✓`, лог дээр `CONSISTENT` гарна (Зураг 7).

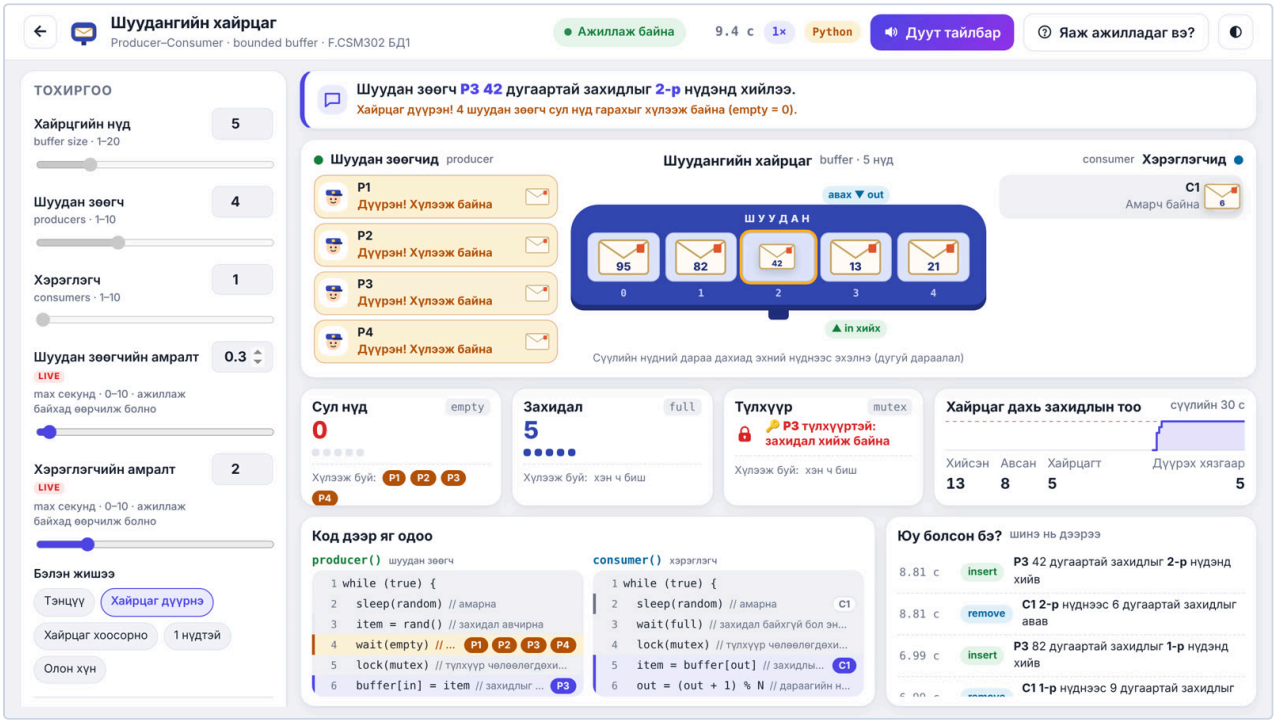


Зураг 7. Жишээ 1. Зогсоосны дараах байдал

Жишээ 2. Буфер дүүрэх (producer sleep 0.2 с, consumer sleep 2 с). Producer-ууд хурдан тул буфер дүүрч `empty = 0` (улаан) болно. Producer-уудын карт шар болж `sem_wait(&empty)` дээр хүлээнэ (Зураг 8). Шуудангийн хуудсанд «Хайрцаг дүүрнэ» бэлэн жишээ ижил байдлыг харуулна (Зураг 9).



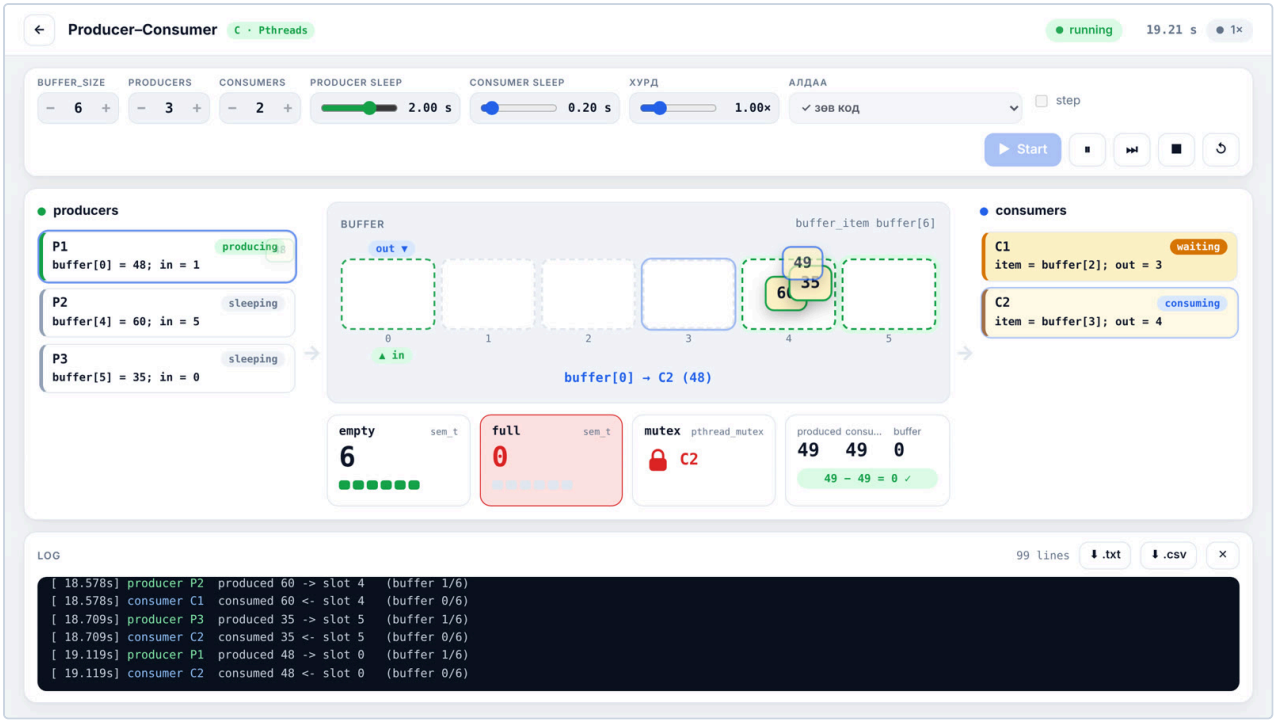
Зураг 8. Жишээ 2. Буфер дүүрсэн: empty = 0



Зураг 9. Жишээ 2. Шуудангийн хуудсанд «Хайрцаг дүүрнэ»: Сул нүд 0, шуудан зөөгчид хүлээж байна

Жишээ 3. Буфер хоосрох (producer sleep 2 с, consumer sleep 0.2 с). Consumer-ууд хурдан тул буфер ихэнхдээ хоосон байж `full = 0` болно. Consumer-ууд `sem_wait(&full)` дээр

хүлээнэ (Зураг 10).



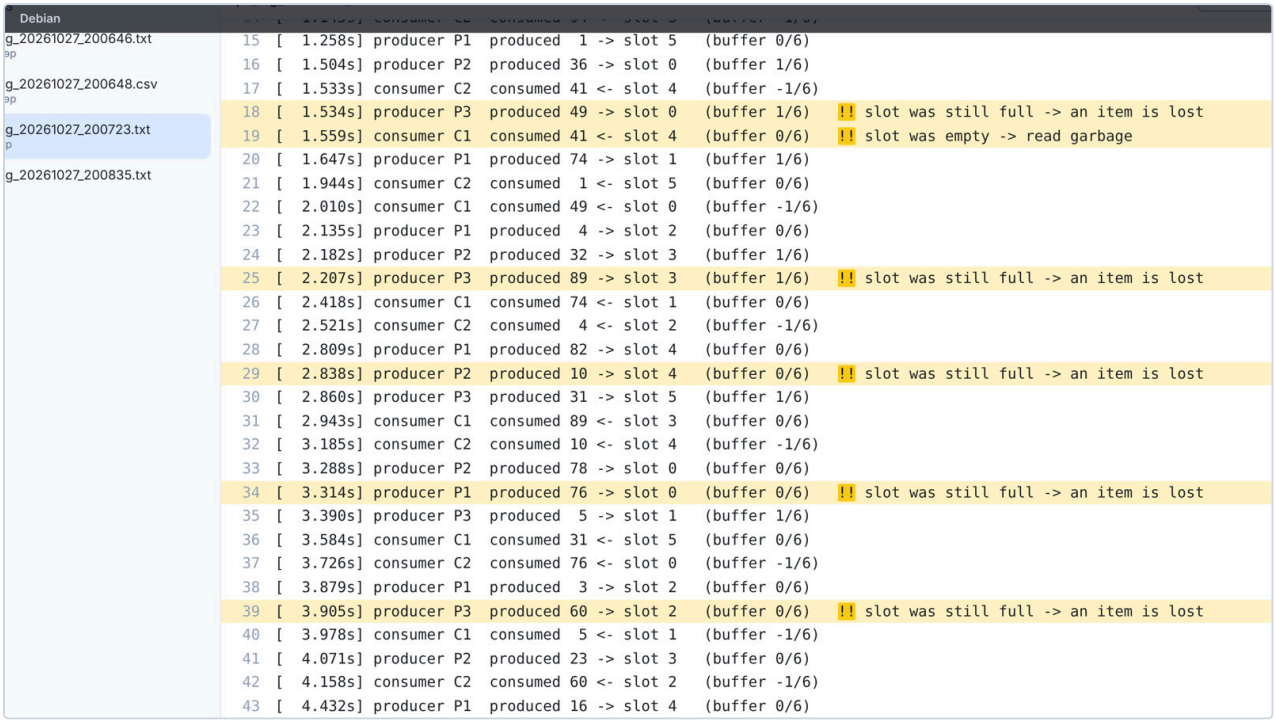
Зураг 10. Жишээ 3. Буфер хоосорсон: full = 0

Жишээ 4. Deadlock (алдаа: lock → wait дараалал). Эхэлмэгц буфер хоосон байхад consumer mutex-ийг аваад `full` хүлээнэ. Producer-ууд mutex авч чадахгүй тул item хэзээ ч орж ирэхгүй (Зураг 6). Stop дарахад лог дээр `DEADLOCK` гарна.

1.13 Лог татах

Лог хэсгийн ⬇ **.txt** товч нь толгой мэдээлэл (огноо, тохиргоо, алдааны горим) бүхий, терминалын гаралттай ижил форматтай текст файл татна. ⬇ **.csv** товч нь Excel / LibreOffice-д нээгдэх хүснэгт татна: `time_s, thread, role, action, item, slot, buffer_count, buffer_size, race`. Хөтөч дээр сүүлийн 600 мөр харагдана. Татах файлд 200 000 хүртэлх мөр орно. Алдааны горимд алдаатай мөр `!!` тэмдэгтэй байна (Зураг 11).

```
# Producer–Consumer · C / Pthreads (src/c/producer_consumer)
# BUFFER_SIZE=6 producers=3 consumers=2 producer_sleep=0.3s consumer_sleep=0.3s
bug=no_mutex
[  1.534s] producer P3  produced 49 -> slot 0   (buffer 1/6)   !! slot was still
full -> an item is lost
[  1.559s] consumer C2  consumed 41 <- slot 4   (buffer 0/6)   !! slot was empty ->
read garbage
...
produced=58 consumed=52 in_buffer=3 -> INCONSISTENT
errors: lost_items=8 bad_reads=5
```



Зураг 11. Тамсан race condition-ийн лог: «!!» хайхад алдаатай мөрүүд тодорсон

1.14 Түгээмэл алдаа ба шийдэл

Хүснэгт 6. Түгээмэл алдаа

Алдаа	Шалтгаан	Шийдэл
`gcc/make олдсонгүй` , C хуудас шар анхааруулгатай	C compiler суугаагүй	`sudo apt install build-essential`
Start дармагц зогсоно (0.00 s), эсвэл «C программ алдаатай зогслоо»	хуучин хөрвүүлэгдсэн программ (zip/git-ээс ирсэн файлын огноо хуучин)	`make -B -C src/c` , эсвэл `gui.py` -г дахин асаах (асах бүрт дахин хөрвүүлнэ)
`… хооронд байх ёстой`	хүрээнээс гадуур утга	Хүснэгт 2-ын хүрээнд оруулна
`Симуляц ажиллаж байна`	ажиллаж байхад дахин Start дарсан	эхлээд Stop эсвэл Reset
Хөтөч нээгдэхгүй	орчинд хөтөч тохируулаагүй	терминалд хэвлэгдсэн хаягийг гараар нээнэ
Дуут тайлбар дуугүй, «зөвхөн хадмал»	аудио файл үүсгээгүй, хөтчид монгол хоолой алга	2.3-ын дагуу `make_voice.py` -оор аудио үүсгэнэ, эсвэл Microsoft Edge ашиглана
`pip install` дээр *externally-managed-environment*	Debian PEP 668	venv үүсгэж суулгана (2.3)
⛔ DEADLOCK, ✕ lost	«алдаа» сонголт идэвхтэй	«✓ зөв код» сонгоод Reset

ХОЁР. OPERATIONAL ЗААВАР

2.1 Repository clone

```
git clone git@github.com:anunbayar1/CSM302-2627-BD1.git
cd CSM302-2627-BD1
```

Repository private тул баг гишүүн SSH түлхүүрээ GitHub-д бүртгэсэн байх шаардлагатай. Эсвэл GitHub дээрээс **Code → Download ZIP** хийж задална.

Хүснэгт 7. Хавтасны бүтэц

Зам	Агуулга
`src/c/`	`buffer.c/.h` (insert_item, remove_item, empty, full, mutex, алдааны туршилт), `main.c` (номын CLI), `ui.c` (вебт төлөв дамжуулах `--ui` горим), `sim.h`, `Makefile`
`src/gui.py`	localhost веб сервер ба API (`/api/c/*`, `/api/py/*`), C программыг удирдана
`src/bounded_buffer.py`	Python хэрэгжүүлэлт (шуудангийн горим)
`src/ui/`	`home.html`, `mail.html`, `raw.html`, `tour.json` (дуут тайлбарын текст), `voice/` (үүсгэсэн аудио)
`tests/`	`test_buffer.py`, `test_c.py`, `test_web.py`
`tools/make_voice.py`	дуут тайлбарын аудио үүсгэгч

2.2 OS, compiler/runtime, version

Хүснэгт 8. Туршсан орчин

Орчин	OS	Compiler	Make	Python
Хөгжүүлэгчийн компьютер	Debian (x86-64)	gcc (Debian)	GNU Make	Python 3
Туршилтын орчин	Ubuntu 24.04.5 LTS (x86-64)	gcc 13.3.0	GNU Make 4.3	3.13

Хөрвүүлэх тохиргоо: `-Wall -Wextra -O2 -std=c11 -D_DEFAULT_SOURCE -pthread`, сануулгагүй хөрвүүлэгдэнэ.

2.3 Dependency суулгах

```
sudo apt install git build-essential python3        # заавал

# сонголттой: дуут тайлбарын аудио (интернет хэрэгтэй)
python3 -m venv ~/.venvs/tts
~/.venvs/tts/bin/pip install edge-tts
~/.venvs/tts/bin/python tools/make_voice.py --preview    # хоолойнуудыг харьцуулах
~/.venvs/tts/bin/python tools/make_voice.py              # 16 алхмын mp3 →
src/ui/voice/
```

Python-ийн гадна сан, Node.js, framework ашиглаагүй. Дуут тайлбарын хоолой (`mn-MN-YesuiNeural`, `mn-MN-BataaNeural`), хурд, өндрийг `--voice`, `--rate`, `--pitch`-ээр солино.

2.4 Build ба Run

```
python3 src/gui.py                          # веб UI (C-г автоматаар build хийнэ)
python3 src/gui.py --port 9000 --no-browser

make -C src/c                               # зөвхөн C программ
make -C src/c BUFFER_SIZE=10                # өөр буферийн хэмжээ
./src/c/producer_consumer 5 3 3             # номын хэлбэр: <унтах сек> <producers>
<consumers>
./src/c/producer_consumer -q 2 8 8 0.01  # -q: зөвхөн эцсийн дүн, [max_nap сек]
./src/c/producer_consumer --bug order 5 3 3   # алдааны туршилт: no-mutex | no-empty
| no-full | order
```

```
$ ./src/c/producer_consumer 3 2 2
BUFFER_SIZE=5 producers=2 consumers=2 run=3.00s max_nap=1.00s
[  0.080s] producer P2  produced  1 -> slot 0   (buffer 1/5)
[  0.191s] consumer C1  consumed  1 <- slot 0   (buffer 0/5)
[  0.403s] producer P1  produced 74 -> slot 1   (buffer 1/5)
...
produced=14 consumed=9 in_buffer=5 -> CONSISTENT
```

Программ `main()` нь номын дагуу: (1) аргумент унших, (2) буфер, семафор, mutex үүсгэх, (3–4) producer, consumer thread үүсгэх, (5) заасан хугацаа унтах, (6) thread-үүдийг зогсоож join хийгээд `produced − consumed = in_buffer` эсэхийг шалгана.

2.5 Үндсэн тестийн өгөгдөл

```
python3 -m unittest discover -s tests     # 42 тест
make -C src/c test                        # CLI: хэвийн, алдааны туршилт, буруу
оролт
make -C src/c tsan                        # ThreadSanitizer
```

Хүснэгт 9. Үндсэн тест (`make -C src/c test`, бодит гаралт)

Команд	Тайлбар	Гаралт
`-q 2 1 1 0.05`	1 producer, 1 consumer	`produced=72 consumed=68 in_buffer=4 -> CONSISTENT`
`-q 2 3 3 0.05`	тэнцүү	`produced=221 consumed=220 in_buffer=1 -> CONSISTENT`
`-q 2 8 1 0.05`	буфер дүүрэх	`produced=79 consumed=74 in_buffer=5 -> CONSISTENT`
`-q 2 1 8 0.05`	буфер хоосрох	`produced=72 consumed=72 in_buffer=0 -> CONSISTENT`
`-q 2 20 20 0`	40 thread, унтахгүй (ачаалал)	`produced=548007 consumed=548005 in_buffer=2 -> CONSISTENT`

Python тестүүд: `test_buffer.py` (12): insert/remove, тойрог дараалал, дүүрэн/хоосон буфер дээр хүлээх, ажиллаж байх үед хурд өөрчлөх. `test_c.py` (6): build, CONSISTENT, буруу аргумент, алдааны 4 горим. `test_web.py` (24): хуудаснууд, start/pause/step/resume/stop/reset, буруу параметр, C ба Python хөдөлгүүрийн тууштай байдал, deadlock илрүүлэлт.

2.6 Нэмэлт boundary ба error тест

Хүснэгт 10. Буруу оролт (бүгд exit 1, usage хэвлэнэ)

Аргумент	Алдааны мэдээлэл
(хоосон), `1 2`	`usage: producer_consumer [-q] [--bug KIND] <sleep_seconds> <producers> <consumers> [max_nap_seconds]`
`0 1 1`, `abc 1 1`	`error: sleep_seconds must be a number in (0, 3600], got '0'`
`1 0 1`	`error: producers must be an integer 1..100, got '0'`
`1 1 101`	`error: consumers must be an integer 1..100, got '101'`
`1 1 1 -1`, `1 1 1 x`	`error: max_nap_seconds must be a number 0..10, got '-1'`
`--bug x 1 1 1`	`error: unknown --bug 'x'`

Хүснэгт 11. Алдааны туршилт (бодит гаралт)

Команд	Гаралт	Exit
`-q --bug no-mutex 1 8 80`	`INCONSISTENT`, `errors: lost_items=3384 bad_reads=3385`	2
`-q --bug no-empty 1 4 1 0.01`	`INCONSISTENT`, `lost_items=586 bad_reads=0`	2
`-q --bug no-full 1 1 4 0.01`	`INCONSISTENT`, `lost_items=0 bad_reads=566`	2
`-q --bug order 2 2 2 0.05`	`produced=0 consumed=0 -> DEADLOCK`, `no progress for the last 2.0s`	3

Веб талд: Python хуудсанд хүрээнээс гадуурх утга (Зураг 4), ажиллаж байхад дахин Start дарах, буруу алдааны төрөл зэргийг сервер HTTP 400 болон монгол мессежээр буцаана (`test_web.py`).

2.7 Хүлээгдсэн үр дүн

- Зөв кодоор хийсэн бүх run-ий төгсгөлд `produced − consumed = in_buffer` → `CONSISTENT`, exit 0. Дэлгэц дээр `✓`.
- Буферт хэзээ ч BUFFER_SIZE-аас олон item байхгүй. Хоосон буферээс уншихгүй. `empty + full` нь BUFFER_SIZE-тэй тэнцүү байна (thread `sem_wait`-ийн дараа, `sem_post`-оос өмнө байх агшинд түр зуур 1–2-оор бага харагдаж болно).
- Алдааны горимд алдаа илэрч, `INCONSISTENT` (exit 2) эсвэл `DEADLOCK` (exit 3) гарна.
- `python3 -m unittest discover -s tests`: `Ran 42 tests … OK`. `make -C src/c test`: `ALL TESTS PASSED`. `make -C src/c tsan`: ThreadSanitizer-ийн анхааруулгагүй, exit 0.

2.8 Known issues

- C хөдөлгүүр зөвхөн Linux / WSL2 дээр ажиллана (`sem_timedwait`, `pthread_mutex_timedlock`). macOS дээр нэргүй семафор (`sem_init`) дэмжигдэхгүй тул туршаагүй.
- Mutex-ийг микросекунд л барьдаг тул дэлгэц дээр сүүлийн эзэмшигчийг ~0.5 с харуулдаг. Энэ нь зөвхөн дүрслэлийн сунгалт.
- Дэлгэц төлөвийг 50–100 мс тутам уншдаг. Нэг удаад ирсэн үйл явдлуудыг нүд бүрт дарааллаар нь тоглуулдаг тул animation бодит хугацаанаас бага зэрэг хоцорно. Хэт хурдан үед зарим нислэгийг алгасна. Тоолуур ба лог үргэлж бодит утгаа харуулна.
- Mutex-гүй туршилтын 40 мс (CLI-д 0.1 мс) завсар нь preemption-ийг дуурайлгах зориулалттай. Бодит программд race condition ийм олон удаа гарахгүй ч боломж нь адилхан.
- ThreadSanitizer нь зөвхөн тухайн ажиллагаанд race илрээгүйг харуулна. Race огт байхгүй гэдгийг баталгаажуулахгүй.

- Дуут тайлбарын аудиог интернеттэй үед урьдчилан үүсгэх шаардлагатай.
- Сервер зөвхөн `127.0.0.1`-д сонсдог (гадаад сүлжээнээс хандах боломжгүй).

2.9 Нөөц ба эх сурвалж

- A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Хэсэг 7.1.1 (The Bounded-Buffer Problem, Figures 7.1, 7.2), Programming Project 4: The Producer–Consumer Problem.
- F.CSM302 Бие даалт 1-ийн удирдамж (OS_BD1_Guide.pdf).
- POSIX: `pthread_create(3)`, `pthread_mutex_timedlock(3)`, `sem_init(3)`, `sem_timedwait(3)` man хуудаснууд.
- Дуут тайлбар: Microsoft Edge TTS-ийн монгол хоолой (`edge-tts`).
- AI ашиглалт: Claude (Anthropic)-г код, тест, UI, баримт бичиг боловсруулахад ашигласан.